

# LLM inference API with batched GPU serving

front: what to ask, what to design · 55 minutes · one problem

**Anthropic questions this is: SD Q1 inference API** (1point3acres, the most frequent card, last asked Sep 11; Tyler read it verbatim): open costume (whole serving stack) or the fixed-batch **batchstring** dispatcher (1–100 strings, ~100 ms per batch fixed, one batch per GPU); follow-ups = 10k QPS at 100 ms → GPU count, the **8-GPU two-model scheduler**, "bring GPU machines up faster" · **onsite doc review** = a flawed version of this design (fixed batching, no rate limiting, no shedding, no retries/idempotency) · **hack2hire** "GPU inference serving system" + "GPU scheduling platform" · **igotanooffer Q1/Q5** chat service and developer APIs put a gateway in front · **Tyler #1** token usage = the gateway's quotas · **Alex mock** ML serving platform = this data plane plus versions and A/B. **Expect:** multi-tenancy and isolation, life of a request.

## 1 REQUIREMENTS 0–8 min · ask, then write sentences

- **Ask:** fixed batch contract or continuous batching? May I autoscale, rate limit, cache, or is this the dispatch layer only?
- **Ask:** chat (TTFT matters) or offline? Client waits or polls? Scale today and at 10×? Which models, how many versions live?
- **A user can:** send a prompt and get tokens back, streaming or in one reply · choose a model and version, old ones included · paid and free tiers, paid first · one API across all replicas.
- **Technical challenges** (say this, not "non-functional"): GPUs near 100% busy · tail latency bounded, P95 500 ms · cold start of inactive models · isolation, no noisy neighbour · availability over consistency · metrics.
- **Anchors:** 100 and 10,000 req/s · batch ≤ 100 · 100 ms per batch fixed · GPU boot is minutes · weights ≈ 1 TB per frontier model.

## 2 CORE ENTITIES 8–12 min · nouns, then fields

- **Request**(id, tenant\_id, model, version, prompt, max\_tokens, stream, idempotency\_key, state)
- **Model**(name) · **ModelVersion**(model, version, min\_gpus, weights\_ref, state active|inactive)
- **Replica**(id, model\_version, gpu\_ids, state warming|ready|draining, queue) — hardware running loaded weights
- **Tenant / Limit**(tenant\_id, tier, rpm, tpm)
- **Batch** is a property of the replica's queue, not an entity. Isolation is a policy on the queues.
- Say aloud: model = blueprint (version + weights on storage); replica = GPUs with the weights loaded; each version needs a minimum GPU count.

## 3 API 12–17 min · one endpoint per requirement

- POST /v1/messages {model, version?, prompt, max\_tokens, stream} → 200 body, or SSE token stream; 429 + retry\_after when shedding
- GET /v1/requests/{id} → state, tokens so far (polling clients)
- GET /v1/models → versions, state, warm replica count
- Internal: POST /replicas/{id}/load {model\_version} · /drain · /warm
- Headers: tenant from auth token, never the body; idempotency key on every POST.

## 4 DATA FLOW 17–22 min · walk one request through

- auth + tenant limit at the gateway → **router** picks the pool by model+version (no live replica → cold-start path) →
- **per-tenant queue** (paid first, fairness) → **per-replica batcher**: flush at N or at 50 ms, whichever first → replica runs prefill then decode →
- finished sequences **leave the batch early** → response service streams tokens → metrics: queue depth, batch fill, TTFT, P99, GPU util.
- Record → batch map kept so a dead replica costs one retry, never a duplicate answer.

## 5 HIGH-LEVEL DESIGN 22–38 min · boxes, one layer per row

- Row 1: clients → API gateway → router → model registry.
- Row 2: queue-depth monitor · per-tenant priority queues · per-replica batcher · scheduler/autoscaler.
- Row 3: replicas (N GPUs + loaded weights + KV cache) per model version; response service on the left.
- Row 4: metrics · warm pool (hardware only) · weights store (NVMe/object, fast load).
- Solid = request path 1–6; dashed = control, cold start, failure. Colour by layer, legend in the corner.
- Capacity out loud:  $100 \div 0.1 \text{ s} = 1,000 \text{ req/s per GPU}$  → 10 GPUs floor at 10k → ~18 at 65% + spares. Budget: 50 batcher + 100 GPU + network, keep ~200 ms for one retry.

## 6 DEEP DIVES 38–50 min · offer, then follow the interviewer

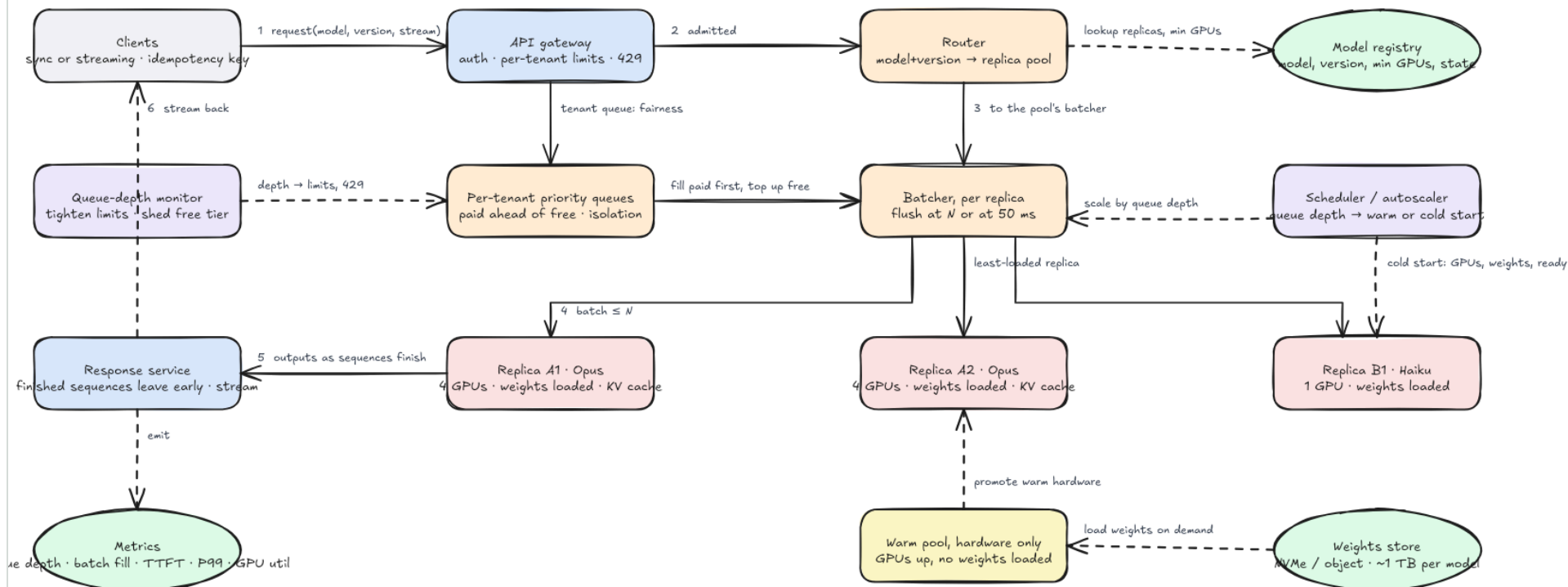
- **Batcher constants** from the budget: timer is the lever at low load, size at high load.
- **Cold start:** warm hardware, never idle weights pinned; scheduler assigns GPUs, streams weights, marks ready; router starts sending. Minutes, so plan for it.
- **Overload:** queue depth is the truth; tighten per-tier limits, shed free, 429 + retry\_after. Not a static token bucket.
- **Isolation:** per-tenant queues, quotas, max tokens per request; one giant request cannot stall the batch.
- **Replica dies mid-batch:** map re-enqueues at head; idempotency key.
- **8 GPUs, two models:** reserve all 8 atomically, drain small batches, age so neither starves, say the idle cost.
- **Cache:** only if the hit rate pays; prefix KV cache for shared system prompts is the one that usually does.

## The board at minute forty

- Bow at 50: decisions, what you would revisit, what you cut. Stop.

LLM inference API with batched GPU serving, the answer

Route by model+version to a replica pool, batch per replica by size-or-time, stream finished sequences back early. Solid = request path, dashed = control, cold start and failure.



Failure: a replica dies mid-batch → the request+batch map re-enqueues those requests at the head of another replica's queue; the client's idempotency key prevents a double answer.  
Cold start: a request for a model with no live replica → scheduler takes warm hardware (up, empty), streams weights from the weights store (~1 TB, minutes), marks the replica ready, router starts sending.  
Keep hardware warm, never the weights of unused models; pinned idle weights block another model from those GPUs.

Legend



Numbers to say out loud

Per replica: batch  $N$  + batch time. Fixed-batch contract 100 per 100 ms  $\rightarrow$  1,000 req/s per GPU  $\rightarrow$  10 GPUs floor at 10k RPS; run at ~65% + spares  $\rightarrow$  ~18.

Budget 500 ms P95:  $\leq 50$  ms in the batcher + 100 ms on the GPU + tens of ms network, keep  $\sim 200$  ms for one retry. At 100 RPS a 50 ms window holds five requests: the timer is the lever.

Weights  $\approx 1$  TB per frontier model, loaded into GPU memory: serving is memory-bound, a model needs a minimum GPU count, and unused weights pinned in memory block another model.

Eight GPUs, two models: reserve all eight atomically for the big one, drain small batches, age so neither starves, say the idle cost.